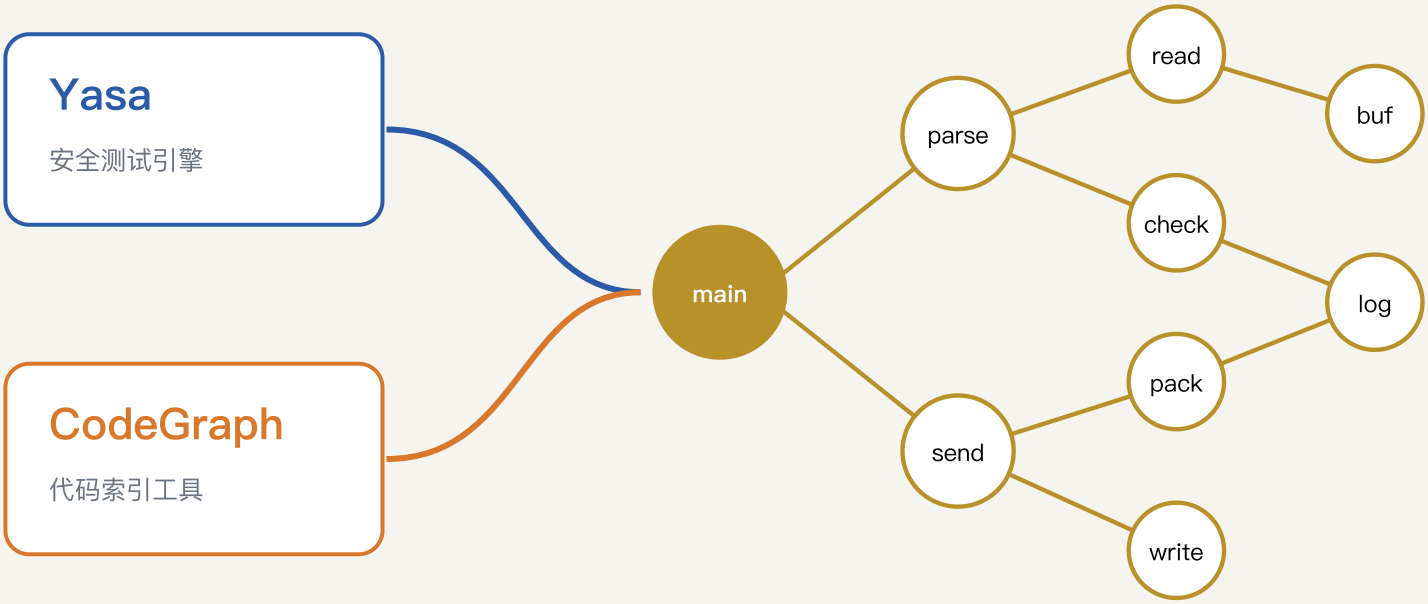
CodeGraph 是什么，以及怎样和 Yasa 比较调用图

介绍开源工具 CodeGraph，说明它和 Yasa 为什么可以放在一起比较、比较的是什么、评测怎样进行。



两者都能输出“谁调用了谁”的调用图

1 CodeGraph 简介

CodeGraph（github.com/colbymchenry/codegraph）是一个开源的代码索引工具，采用 MIT 协议，2026 年 1 月发布，到 2026 年 11 月初在 GitHub 上约有 7.7 万 star。它扫描整个仓库，把函数、类、文件以及它们之间的调用、引用和继承关系存进本地数据库，再提供给 Claude Code、Cursor 等 AI 编程助手查询。

它要解决的问题是：AI 助手在回答“这个功能是怎么实现的”“改这个函数会影响哪里”之类的问题时，往往要一个个打开文件去读。有了索引，助手可以直接查到调用关系和相关代码。



图 1 典型使用场景（示意）。助手通过 MCP 协议查询 CodeGraph 的索引。



图 2 CodeGraph 的处理流程。抽取过程完全由语法树决定，不经过大模型。

直接调用：按名字找，再用规则筛

遇到 `foo()` 这样的调用，CodeGraph 先找出项目里所有叫 `foo` 的定义，再逐个筛选和打分。C 语言的 `static` 函数只在本文件可见，会被直接排除；剩下的候选里，同一文件的定义得分最高，其次看目录远近、语言是否一致等。同名定义多到没法判断时，它会放弃匹配，而不是随便选一个。以上规则均对照 v1.6.2 源码确认。

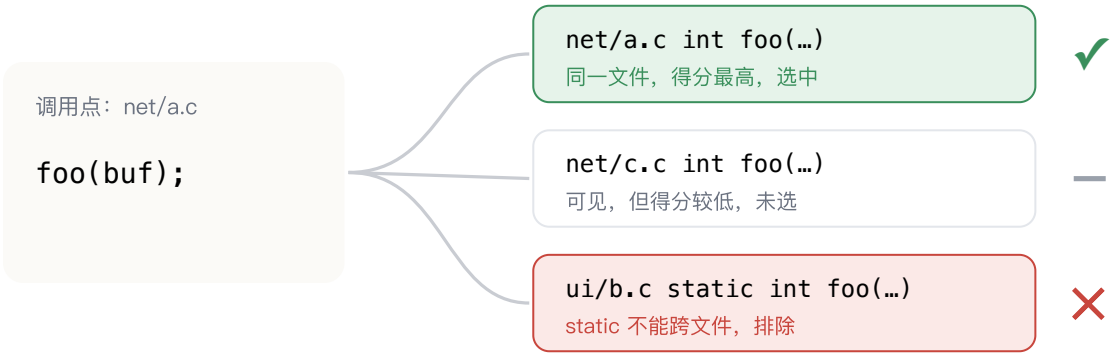


图 3 一个调用有多个同名候选时的选择过程（示意）。

函数指针调用：按“结构体类型 + 字段”找

C 语言里常见 `p->fn()` 这种通过结构体字段调用函数的写法，调用点上没有函数名。CodeGraph 的做法是：先在全项目中找出所有“把某个函数赋给某类结构体的 `fn` 字段”的地方，包括初始化列表、`.fn = f` 和 `x->fn = f`，以及宏生成的函数表；调用时再按 `p` 的结构体类型，把这些函数都连上。这一部分直接分析源码文本，宏和条件编译由工具自己模拟，得到的边被标记为“启发式”。



图 4 函数指针按“结构体类型 + 字段”汇总（示意代码）。

索引里记录了什么

索引的核心是两张表：`nodes` 记录每个函数、类、变量的名字、所在文件和起止行号；`edges` 记录它们之间的关系，调用关系的类型是 `calls`。每条边还带一个来源标记，取值为 `tree-sitter`、`scip` 或 `heuristic`，函数指针推断出来的边属于最后一种。

nodes（节选）

id	kind	name	file_path
n1	function	dispatch	core.c
n2	function	read_line	util.c
n3	function	do_read	io.c
n4	struct	ops	io.h

edges（节选）

source	target	kind	line	provenance
n1	n2	calls	12	tree-sitter
n1	n3	calls	15	heuristic

第二条是通过 p->fn() 推断出来的回调

图 5 索引中的节点和调用边（示意数据，字段名取自 v1.6.2 的数据库结构）。

CodeGraph 不做的事

它不做漏洞检测，没有污点分析，也没有漏洞规则。它提供给 AI 助手的 8 个工具都是代码查询：搜索、查调用者、查被调用者、影响范围、节点详情、流程探索、索引状态和文件列表。

2 Yasa 简介

Yasa 是蚂蚁集团开源的静态应用安全测试引擎，用来发现代码中的安全漏洞，支持多种语言，C 语言支持正在开发中。它先把源代码转换成统一抽象语法树（UAST），C 代码在这一步按项目的编译配置展开宏、处理条件编译；然后对程序做符号执行，跟踪变量、指针和结构体字段的取值，同时生成调用图；最后在调用图上做污点分析，输出漏洞报告。



图 6 Yasa 的处理流程。调用图是中间产物，漏洞检测建立在它之上。

“按编译配置”意味着什么

C 项目大量使用条件编译。同一份源码，在不同平台、不同编译选项下，实际参与编译的代码并不一样。编译器只会选其中一支；只看源码文本的工具，需要自己判断哪一支生效。Yasa 在生成 UAST 时就按给定的编译配置做了这个选择。

```
int open_file(const char *p) {
#ifdef _WIN32
    return win_open(p);
#else
    return posix_open(p);
#endif }
```

在 Linux 配置下

open_file → posix_open　参与编译

open_file → win_open　不参与编译

图 7 条件编译的例子：真实的调用图取决于编译配置。

两种做法的区别

	CodeGraph	Yasa
用途	为 AI 助手提供代码索引	发现安全漏洞
代码解析	tree-sitter 语法树	统一抽象语法树（UAST）
宏和条件编译	部分由工具自行模拟	按编译配置展开和取舍
调用关系	名字匹配加规则；函数指针按类型和字段汇总	符号执行中跟踪实际取值
漏洞检测	无	有（污点分析）

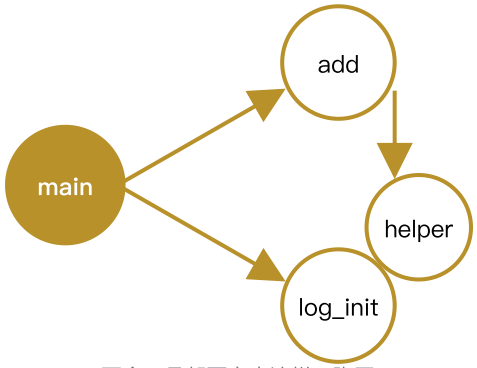
3 为什么可以对比

两个工具的用途不同，但都能输出同一种东西：程序的调用图，也就是“哪个函数调用了哪个函数”。把两边的输出整理成相同的格式，就可以用同一份标准答案逐条核对。

```
// util.c
static int helper(int x)
{ return x + 1; }

int add(int a, int b)
{ return helper(a) + b; }

int main(void)
{ log_init(); return add(1, 2); }
```



两个工具都要交出这样一张图

图 8 一段代码和它的调用图。评分时逐条检查每条边对不对、标准答案里的边有没有找到。

把两边的输出变成同一种格式

两个工具的原始输出长得不一样：CodeGraph 是数据库里的两张表，Yasa 是一个 JSON 文件，节点名里带着文件和行号。评测前要把它们都转换成同一种记录，每条记录说明“哪个文件的哪个函数，在第几行，调用了哪个文件的哪个函数”。

CodeGraph：edges + nodes 表

```
source=n7 target=n9
line=3 (n7=add, n9=helper)
```

Yasa：callgraph.json

```
"add [util.c : 3_3]" ->
"helper [util.c : 2_2]"
```

统一后的记录

调用方 util.c::add
被调方 util.c::helper
位置 util.c 第 3 行

图 9 两种原始输出转换成统一的记录（示意数据）。标准答案也整理成同样的格式。

为什么选 CodeGraph

使用广泛

目前是同类工具中最受关注的一个，用它作参照有代表性。

语言覆盖

支持 C、C++、Rust 等语言，以后评测其他语言可以沿用同一套方法。

结果可导出

索引保存在本地 SQLite 数据库里，可以完整导出每一条调用边。

比较范围

只比较调用图，不比较漏洞检测。CodeGraph 没有这项能力。

4 评分口径

标准答案由编译器给出

选一个真实的开源 C 项目，按它自己的构建配置编译通过，再用 clang 分析每一个参与编译的源文件，列出所有函数和函数之间的每一处直接调用。编译器知道宏怎样展开、`#if` 选了哪一支、同名函数具体是哪一个，所以它给出的直接调用可以作为标准答案。

准确率和召回率

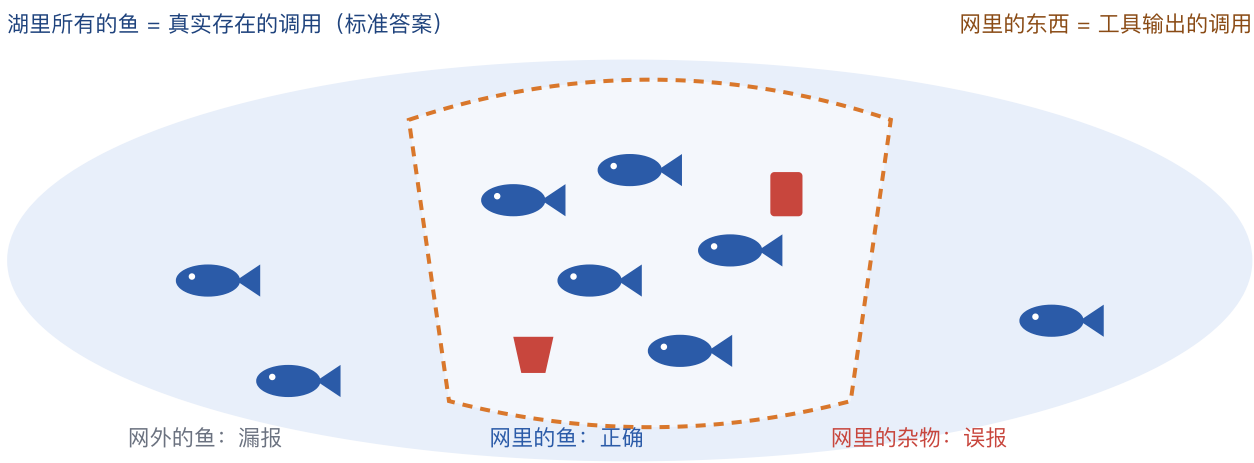


图 10 用撒网捕鱼来理解两个指标。

指标	问的是	算法
准确率（精确率）	网里捞上来的东西，有多少是鱼	正确 ÷ 工具输出总数
召回率	湖里的鱼，有多少被捞了上来	正确 ÷ 标准答案总数

两个指标要一起看。网撒得越大，捞到的鱼越多，杂物也越多；只捞最有把握的几条，杂物少了，漏掉的鱼又多了。

三类调用分开统计

不含宏的直接调用

```
send(buf);
```

最基础的调用，可以逐条核对

算准确率和召回率

全部直接调用

```
CHECK(len);
```

宏展开后才出现的调用也算
标准答案里含编译期辅助代码

只算召回率

函数指针回调

```
p->fn(arg);
```

编译器也给不出目标
由人工审定一批真实样本

只算样本召回率，不代表全项目

图 11 三类调用的统计口径不同。

5 对比流程



图 12 对比流程。第 2 步的答案在看到任何工具输出之前确定，并保存校验值，之后不再修改。

核对时要注意的地方

net/a.c

```
static int helper()
void a(){ helper(); }
```

两个不同的 helper

a.c::helper 和 b.c::helper

只写 helper 会混在一起

ui/b.c

```
static int helper()
void b(){ helper(); }
```

图 13 同名的 static 函数。核对时要用“文件 + 函数名”作为标识。

宏

```
#define LOG(m) log_write(m)
LOG("start");
```

源码里写的是 LOG，
实际调用的是 log_write。

“不含宏”和“含宏”分开统计，
才能看清差距从哪里来

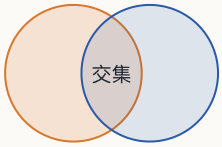
条件编译

```
#if 0
old_init();
#endif
```

这段代码不会被编译。

工具如果连到 old_init，
按误报计

评分范围



只在两个工具都认得的
函数上打分，难题会被排除。

范围要事先固定

图 14 另外三个需要注意的地方（示意代码）。

6 函数指针回调的评估

回调的目标在调用点上看不出来，编译器也不知道，因此没有现成的标准答案。常用的处理方法是第 1 节介绍的“按结构体类型 + 字段”汇总，CodeGraph 就是这样做的；Yasa 也可以采用同样的汇总方式，区别在于它基于按编译配置展开过宏的语法树和类型信息。

这种方法有一个固有的局限：同一类型的不同对象会被合并在一起。下面是一个例子。

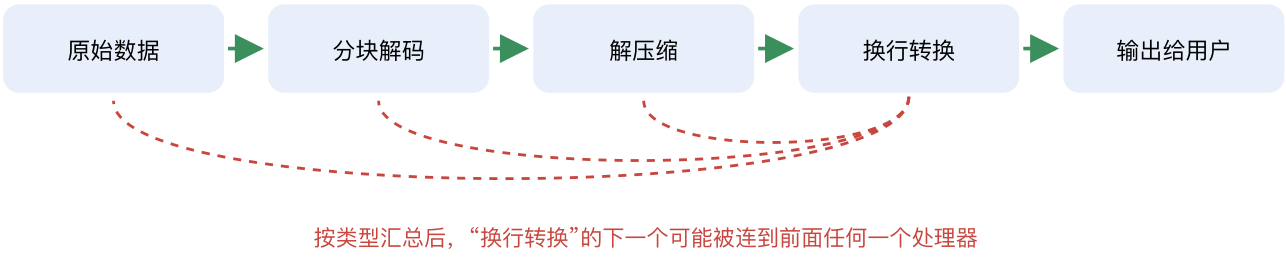


图 15 一条按固定顺序排列的处理链（示意）。每个处理器的下一个只可能是后面的某一个，但它们是同一种结构体类型，按类型汇总无法区分前后。

回调怎样打分

回调的召回率依赖人工审定的样本。每个样本是一个真实的回调调用点，审定人员读代码，确定这个点在当前编译配置下可能调用的全部函数，然后冻结。工具的输出拿来逐个对照：审定目标里找到了几个、漏了几个；样本以外多连的目标不计入召回率，留给调用点抽查去判断对错。



图 16 一个回调样本的对照方式（示意代码和示意结果）。这个样本的召回是 4 个目标中找到 3 个。

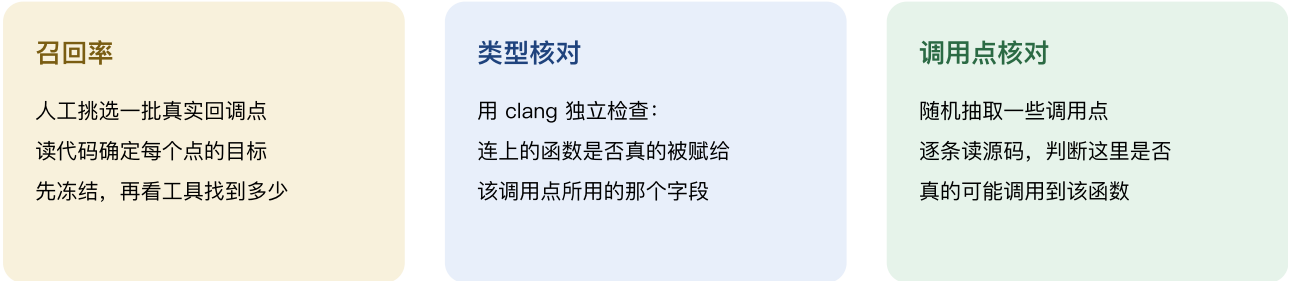


图 17 回调的三种检查。类型核对能发现实现错误，调用点核对才能估计准确率。

7 调用图和漏洞检测的关系

这次只比较调用图，但调用图会直接影响 Yasa 的漏洞检测。Yasa 做污点分析时沿着调用关系追踪数据：用户输入从哪里进来，经过哪些函数，最后是否到达危险函数。调用图如果少了一条边，追踪就在那里中断，后面的危险调用也就发现不了。

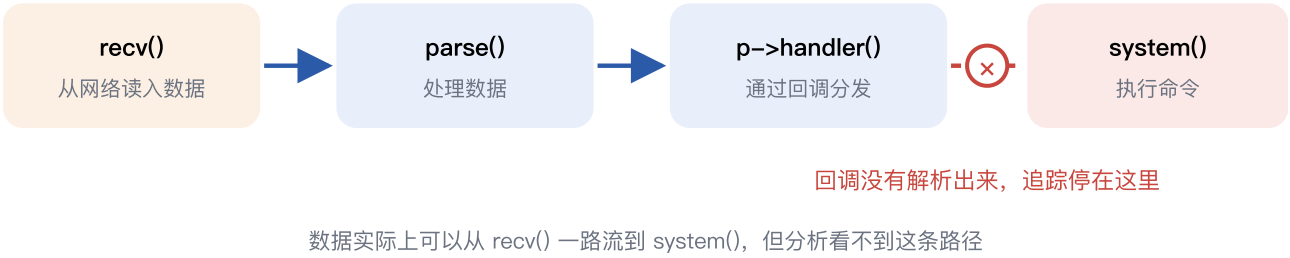


图 18 调用图缺边导致污点追踪中断（示意）。

因此，调用图是否完整、准确，是漏洞检测效果的基础之一。不过调用图做得好，并不等于漏洞检测做得好。如果要评测漏洞检测能力，应使用 Juliet、CASTLE 这类漏洞测试集，对照工具也应换成 CodeQL、Semgrep 等漏洞检测工具。

同样的调用图评测方法也可以用于 Rust、C++ 等语言，前提是 Yasa 已经支持相应的语言。

小结

问题	回答
CodeGraph 是什么	开源的代码索引工具，把函数和调用关系存进本地数据库，供 AI 编程助手查询。不做漏洞检测。
为什么能和 Yasa 比	两者都能输出调用图，可以整理成同一种格式，用同一份标准答案核对。
比的是什么	调用图的准确率和召回率，按不含宏的直接调用、全部直接调用、函数指针回调三类分别统计。
怎么比	同一项目、同一配置；clang 给出直接调用的标准答案，回调由人工审定样本；答案先冻结，再逐条核对，最后人工抽查。

参考：CodeGraph 开源仓库 colbymchenry/codegraph，v1.6.2 版本源码及项目说明。文中示意代码为说明用途编写或改写，不对应具体项目的完整实现。